



**ДЪРЖАВЕН ИЗПИТ
ЗА ПРИДОБИВАНЕ НА ТРЕТА СТЕПЕН НА ПРОФЕСИОНАЛНА
КВАЛИФИКАЦИЯ – ЧАСТ ПО ТЕОРИЯ НА ПРОФЕСИЯТА**

ДИПЛОМЕН ПРОЕКТ

Тема: „Разработване на Android мобилно приложение за следене на здравето на човек“

**професия: 481020 „Системен програмист“
специалност: 4810201 „Системно програмиране“**

**Дипломант: Максим Спасов Ненчев
Ръководител на дипломния проект: Ивайло Абаджиев**

Подпис:
(дипломант)
Подпис:
(ръководител)

София, 2024 г.



Съдържание

1. Въведение	4
1.1. Анализ на заданието.....	4
1.2. Основна идея на заданието	4
2. Основна част	5
2.1. Технологии.....	5
2.1.1 Избор на технологии за база данни	5
2.1.1.0 Разновидност на вида база данни	5
2.1.1.1 Firebase Realtime Database.....	7
2.1.1.2 RoomDB	8
2.1.2 Front-end езици.....	10
2.1.2.1 XML	10
2.1.3 Back-end езици	11
2.1.3.1 Kotlin	11
2.1.3.2 SQLite	12
2.1.4 Използвани сензори и услуги	13
2.1.4.1 TYPE_STEP_SENSOR.....	13
2.1.4.2 GMS LocationServices	14
2.1.4.3 NOTIFICATION_SERVICE.....	15
2.2. Софтуерен дизайн.....	16
2.2.1 Софтуерна архитектура – MVVM	16
2.2.1.1 Model.....	17
2.2.1.2 View (UI Layer)	18
2.2.1.3 View Model	19
2.2.1.4 Service layer	21
2.2.2 Дизайн на базата данни	23
2.3. Удостоверяване на потребителя.....	25
2.4. CRUD заявки.....	25
2.4.1 Create	26
2.4.1.1 Create в RoomDB.....	26
2.4.1.2 Create във Firebase Realtime Database	27
2.4.2 Read.....	28
2.4.2.1 Read в RoomDB	28
2.4.2.2 Read във Firebase Realtime Database	29
2.4.3 Update.....	29
2.4.3.1 Update в RoomDB.....	30
2.4.3.2 Update във Firebase Realtime Database	30
2.4.4 Delete	31
2.4.4.1 Delete в RoomDB.....	31
2.4.4.2 Delete във Firebase Realtime Database	31
3. Функционалности на приложението	32
3.1. Регистриране и влизане – Login Activity.....	32
3.2. Екрани на MainActivity	32
3.2.1 Home	33



СОФИЙСКА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО ЕЛЕКТРОНИКА "ДЖОН АТАНАСОВ"
„Разработване на Android мобилно приложение за следене на здравето на човек“
Максим Спасов Ненчев

3.2.2 Dashboard	33
3.2.3 Account	33
3.2.4 Background функционалности	34
3.2.4.1 StepCounterService.....	34
3.2.4.2 SpeederService	34
3.2.4.3 NotificationService.....	35
4. Заключение	35
4.1. Постигнати цели.....	35
4.2. Развитие.....	35
5. Източници	36
6. Приложение	38



1. Въведение

1.1. Анализ на заданието

Проектът ще представлява реализация на мобилно приложение за операционната система „Android“, за следене на здравето и активността на потребителя във всекидневния му живот.

Всеки потребител ще има собствен акаунт чиито данни ще бъдат записвани локално във устройството му и периодично предавани към облачно пространство за запазване на данни. Акаунта се създава и персонализира лично от потребителя, като паролата му се хешира и единствено той може да я знае или при нужда - промени чрез имейл. Потребителя ще има възможност да записва данни за себе си от всеки изминал ден, включвайки тегло, изпита вода, продължителност на съня и други. Автоматично ще се измерват и изчисляват данни като извървени крачки или изгорени калории и периодично ще се записват в локалната база данни. Всеки ден, данните на потребителя от текущия ден ще бъдат прехвърлени в обект, отговарящ за данните от дните, в които приложението е било инсталирано и активно в телефона. Тези данни ще бъдат представени на потребителя чрез различни таблици и графики със записаната информация. Потребителя ще има собствен лист с приятели, към които ще може да изпраща, и от които ще може да приема шаблонни предизвикателства, свързани с активна дейност. Предизвикателствата се задават с времетраене и вид (тичане, каране на колело и др.), като след задаването им, съществуват и могат да бъдат приети само в деня в който са зададени. Потребителите могат да задават персонализирани цели за физическа активност в съответствие с техните индивидуални нужди и желания, като приложението им предоставя редовни обновления за напредъка им.

Приложението ще използва сензори на устройството, като сензор за броене на стъпки, сензор за ускорение, както и разрешения, позволени от потребителя като право за известия, право за следене на локация, работа на приложението на заден план и други.

1.2. Основна идея на заданието

Във времена, когато модерният начин на живот ни подлага на продължителни периоди на седене и ограничава нашата физическа активност, неотложната необходимост от инструменти за подкрепа на здравословния начин на живот става още по-ясна. Приложението е отговорът на тази нужда. Създадено с цел да бори с проблема на изчезващата физическа активност в нашия ежедневен живот. Приложението представлява



интелигентен компаньон, който помага на потребителите да изграждат по-активен начин на живот.

То се фокусира върху преодоляването на предизвикателствата, свързани с изчезващата физическа активност, като предлага интуитивен и лесен за използване интерфейс, персонализирани цели за физическа активност и непрекъснато следене на потребителския напредък. С помощта на приложението, потребителите се подканват и мотивират да въведат повече движение и здравословни дейности в техния ежедневен ритъм. Съвременните приложения за активност, от които най-известното за Android - Samsung Health, въпреки опитите си за създаване на активна общност около придържането на активен живот, притежават много безличен, самостоятелен и застоял начин на събиране на потребителите си и задействането им в активна дейност.

Приложението създава по-приятелска алтернатива на вече създадените приложения за следене на активност. То ще позволява на потребителя да търси приятели чрез имената им, контрастиращо с други, където събирането на приятели може да се извърши единствено чрез кодове или ако бъдат намерени в конкурси като месечна ранглиста.

2. Основна част

2.1. Технологии

2.1.1 Избор на технологии за база данни

Избирането на база данни за Android приложение е ключова стъпка, която определя как данните ще бъдат управлявани и обработвани в приложението. Вместо да се придържа към традиционните релационни бази данни, проекта използва Firebase Realtime Database като облачна база данни и RoomDB като локална база данни, като се работи с обекти. Ето някои от основните причини за този избор:

2.1.1.0 Разновидност на вида база данни

2.1.1.0.1 NoSQL база данни

NoSQL базите данни са разновидност от бази данни, които се различават от традиционните релационни бази данни (SQL) по редица аспекти. Вместо да се придържат към стандартната таблична структура, характерна за SQL базите данни, NoSQL базите данни



се характеризират с гъвкава схема на данните и обикновено предоставят по-широк набор от възможности за съхранение и обработка на данни.

Гъвкава схема на данните:

За разлика от SQL базите данни, които изискват предварително дефинирана схема за структуриране на данните, NoSQL базите данни позволяват гъвкава схема на данните. Това означава, че не е необходимо да се дефинират всички полета предварително и могат да бъдат добавяни нови полета без прекъсване на работата на приложението.

Поддръжка на големи обеми данни:

NoSQL базите данни често се използват за съхранение на големи обеми данни, с които не се справят добре традиционните SQL бази данни. NoSQL предлагат по-голяма мащабируемост и производителност при работа с големи данни.

Подход към различни типове данни:

NoSQL базите данни поддържат различни типове данни, включително структурирани, полуструктурирани и неструктурирани данни. Това прави тяхната селекция подходяща за различни приложения, като социални мрежи, анализ на данни, уеб приложения и други.

Хоризонтална мащабируемост:

Много NoSQL бази данни предлагат възможност за хоризонтално мащабиране, което позволява лесно добавяне на нови сървъри или кластери за увеличаване на капацитета и производителността на системата.

Подходящи за дистрибуирана среда:

NoSQL базите данни са често използвани в дистрибуирани среди, където е необходимо разпределено съхранение на данни и работа с множество разпределени сървъри или обекти.

Тези характеристики правят NoSQL базите данни предпочитан избор за множество приложения, които изискват гъвкавост, мащабируемост и производителност при работа с големи обеми данни. Те се използват широко в сфери като уеб разработка, анализ на данни и други.



2.1.1.0.2 ORM

ORM (Object-Relational Mapping) е техника за програмиране, която се използва за преобразуване на данни между обектно-ориентирания модел на програмиране и релационната база данни. Всъщност, ORM се използва за улесняване на работата с бази данни чрез използване на обекти в програмния код, вместо таблици и SQL заявки.

Основната цел на ORM е да улесни и ускори разработката на софтуерни приложения, като предоставя абстракция между програмния код и базата данни. Вместо да се пишат SQL заявки и да се манипулират резултатите, може да се работи с обекти в програмния код, които автоматично се преобразуват в съответни записи в базата данни и обратно.

ORM използва мапиране на обекти към редове в таблиците на базата данни и предоставя удобен начин за извършване на операции като създаване, четене, обновяване и изтриване (CRUD) на данни. Това позволява на разработчиците да се фокусират върху логиката на приложението, вместо да се занимават с подробности на базата данни.

ORM библиотеки предоставят различни функционалности и възможности за конфигурация, като поддръжка на връзки между обекти, заявки за данни, транзакции и други. Някои от популярните ORM библиотеки включват Hibernate за Java, Entity Framework за .NET, Django ORM за Python и Room за Android.

Използването на ORM улеснява поддръжката и разширяемостта на приложението, като предоставя структуриран и удобен начин за работа с база данни. Това прави ORM важен инструмент за разработка на софтуер, особено за приложения, които използват релационни бази данни и обектно-ориентиран подход в програмния код.

2.1.1.1 Firebase Realtime Database

Firebase Realtime Database предоставя удобство и ефективност при реалновременната синхронизация на данни между различни устройства и сървъри. Изборът на Firebase Realtime Database за приложението е мотивиран от няколко ключови фактора, които водят до неговото предимство пред други облачни бази данни:

Лесна интеграция с Android приложенията: Firebase Realtime Database е разработен от Google и е специално оптимизиран за интеграция с Android приложения. Той предоставя SDK, което значително улеснява процеса на връзка между приложението и базата данни.



Това ускорява разработката и позволява на разработчиците да се съсредоточат върху функционалността на приложението, вместо да губят време върху сложни конфигурации.

Синхронизация на данни в реално време: Едно от ключовите предимства на Firebase Realtime Database е неговата способност да осигури синхронизация в реално време на данни между различни устройства и сървъри. Това е особено полезно за многопотребителски приложения или за приложения, които изискват бърз обмен на данни.

Управление на инфраструктурата от страна на Google: Firebase управлява инфраструктурата за база данни, което означава, че не се налага да се грижите за сървъри и инфраструктура. Това намалява операционните разходи и освобождава време и ресурси за разработка на функционалността на приложението.

Гъвкавост и мащабируемост: Въпреки че Firebase Realtime Database е база данни на NoSQL модел, тя предоставя голяма гъвкавост и мащабируемост, които са от съществено значение за разработката на Android приложения. Тя позволява лесно добавяне на нови данни и промени в структурата на данните без прекъсване на работата на приложението.

Интеграция на Android Studio с Firebase: Android Studio има вградена поддръжка за интеграция с Firebase, която прави много лесно свързването на приложението с Firebase услугите. Firebase се добавя лесно към проекта директно от Android Studio, като се следват няколко прости стъпки, и след това директно може да се използват Firebase API-тата за взаимодействие с различните услуги.

Съчетанието от лесна интеграция, реалновременна синхронизация, управление на инфраструктурата и гъвкавост прави Firebase Realtime Database предпочитан избор за разработка на Android приложения, включително това. Той отговаря на изискванията за ефективно управление на данните и осигурява отлично потребителско изживяване за крайните потребители.

2.1.1.2 RoomDB

Изборът на RoomDB като локална база данни за приложението е обоснован от няколко ключови фактора:



Интеграция с Android приложенията:

RoomDB е част от Android Jetpack и е специално проектиран за работа с Android приложения. Това означава, че се вписва перфектно в Android архитектурата на приложението и се интегрира лесно с другите компоненти на приложението.

Удобство и лесно използване:

RoomDB предоставя удобен и декларативен начин за работа с локални данни. Той използва анотации и компилира време на аспекти, което прави работата с данните по-проста и удобна за разработчиците.

Съвместимост със SQLite:

RoomDB е построен върху SQLite, което е вградената база данни в Android. Това означава, че RoomDB предлага всички функционалности на SQLite, но с добавена абстракция и улеснен API.

Поддръжка на ORM (Обектно-релационно преобразуване):

RoomDB предлага ORM функционалности, което позволява лесно мапиране на обектите на Java/Kotlin към редове в базата данни. Това улеснява работата с данните и намалява нуждата от ръчно писане на SQL заявки. Именно чрез тези обекти се прави синхронизирането на локалната и облачната база данни.

Сигурност и стабилност:

RoomDB предоставя сигурно и стабилно съхранение на данни на устройството. Той използва проверени технологии и алгоритми за гарантиране на цялостта на данните и предпазване от загуба или повреда.

Съчетанието от удобство, лесна интеграция, съвместимост със SQLite и сигурност прави RoomDB предпочитан избор за локална база данни в Android приложения. Той отговаря на изискванията за ефективно управление на данните на устройството и предоставя отлично потребителско изживяване за крайните потребители.



2.1.2 Front-end езици

2.1.2.1 XML

XML (Extensible Markup Language) е език за маркиране, който се използва широко за описание на структурирани данни. В контекста на разработката на Android приложения, XML се използва предимно за деклариране на потребителски интерфейси (UI) и ресурси като изображения, струни и други.

Интерфейс на потребителя (UI):

В Android разработката, XML се използва главно за деклариране на потребителски интерфейс (UI). Файловете XML, наречени "layout" файлове, съдържат описание на разположението и структурата на елементите в потребителския интерфейс на приложението, като например бутони, текстови полета, изгледи и други.

Дефиниране на ресурси:

XML файловете се използват и за дефиниране на различни ресурси в Android приложението, като например цветове, струни, изображения и други. Тези файлове, наречени "resource" файлове, съдържат данни, които се използват в приложението и се съхраняват в папките "res/values", "res/drawable", "res/mipmap" и други в рамките на проекта. Поддръжка в Android Studio:

Android Studio предлага богата поддръжка за XML файлове. Редакторът на XML в Android Studio предлага автоматично допълване на кода, проверка на синтаксиса и дори визуално представяне на разположението на елементите в потребителския интерфейс чрез вграден редактор за изглед на дизайна. Това прави разработката на потребителски интерфейси по-лесна и интуитивна.

Интеграция с Java/Kotlin код:

XML файловете се използват за деклариране на потребителския интерфейс и ресурсите в Android приложението, а Java или Kotlin кодът се използва за програмирането на логиката зад интерфейса. Android Studio предоставя интеграция между XML файловете и Java/Kotlin кода, което позволява лесна комуникация между тях и улеснява разработката на приложението като цяло.

Манифест файл (AndroidManifest.xml):



Всеки Android проект съдържа манифест файл, който съдържа основната информация за приложението. Това включва декларации за различни компоненти на приложението като activity файлове, service файлове, приемници на съобщения, разрешения от потребителя и други. Манифест файлът предоставя информация за разрешенията, които приложението изисква, и декларации за хардуерни и софтуерни изисквания. Този файл се намира в основната директория на проекта.

Меню файлове:

XML файловете се използват и за дефиниране на менюта в Android приложението. Те могат да съдържат декларации за опции на менюта, контекстни менюта, навигационни менюта и други. Меню файловете се намират в папката "res/menu" в рамките на проекта.

Анимации и анимационни списъци:

XML файловете могат да се използват за дефиниране на анимации и анимационни списъци в Android приложението. Те могат да съдържат декларации за различни видове анимации като въртене, мащабиране, преместване и други. Тези файлове се намират в папките "res/anim" и "res/animator" в рамките на проекта.

2.1.3 Back-end езици

2.1.3.1 Kotlin

За Front-end език е избран Kotlin. Той е мощен и съвременен език за програмиране, който става все по-популярен в света на разработката на Android приложения. Kotlin предоставя изобилията от предимства:

Интероперативност с Java:

Kotlin е проектиран да бъде напълно съвместим с Java. Това означава, че може да се използва съществуващ Java код в Kotlin проекта без проблеми, като могат да се използват източници и Java Code Snippets, без ръчно да бъдат пренаписвани на Kotlin. Освен това, Android Studio – средата, използвана за изработка на приложението може автоматично да превръща Java кода към Kotlin, което съответно улеснява работата с legacy code при нужда за интегриране на чужд код. В действителност, Kotlin код може да бъде включен дори в същия файл с Java код.



Безопасен и изразителен:

Kotlin предлага редица функции, които подобряват безопасността на кода. Например, системата за типове в Kotlin поддържа типизиране по подразбиране за нулеви стойности, което намалява вероятността от NullPointerException грешки. Той всъщност е много фокусиран около null-pointer-и – притежава elvis оператор, позволяващ на разработчика да действа и пише по-бързо и лесно NullChecks и да оперира спрямо получените стойности. Освен това, Kotlin предлага по-кратък и четим синтаксис, който прави кода по-разбираем и лесен за поддръжка.

Автоматично допълване на кода и интегрирана документация:

Android Studio предлага автоматично допълване на кода за Kotlin, което значително улеснява процеса на писане на код и намалява вероятността за грешки. Освен това, вградената документация в Android Studio предоставя богато описание и примери за използване на различните функции и класове в Kotlin, което прави ученето и използването на езика много по-лесно.

Инструменти за отстраняване на грешки и профилиране на приложенията:

Android Studio предлага богата гама от инструменти за отстраняване на грешки и профилиране на приложенията, които помагат да се намерят и отстранят грешките в приложението и да се оптимизира неговата производителност. Тези инструменти са особено полезни при разработката на големи и сложни приложения в Kotlin.

2.1.3.2 SQLite

SQLite се използва в RoomDB като вградена база данни, която предоставя мощни възможности за работа с данни в Android приложенията. SQLite е вградена в повечето мобилни устройства, включително Android смартфони и таблети. Това означава, че не е необходима отделна инсталация или конфигурация за използване на SQLite приложението. SQLite е лека база данни, която е изключително бърза за работа с малки и средни количества данни. Тя е оптимизирана за използване на устройства с ограничени ресурси като мобилни телефони.

Създаване на същност (Entity):

Първата стъпка е да създадете клас, който представлява таблица в базата данни. Този клас обикновено е анотиран с анотация @Entity, която му указва да бъде запазен в базата данни. Всяко поле в този клас обикновено съответства на колона в таблицата.



Дефиниране на Data Access Object (DAO):

След като се дефинира същността, се създава интерфейс, наречен DAO, който съдържа методи за взаимодействие с базата данни. Тези методи могат да включват заявки за вмъкване, актуализиране, изтриване и извличане на данни от базата данни.

Създаване на база данни (Database):

С RoomDB трябва да създадете абстрактен клас, който разширява класа RoomDatabase. Този клас трябва да съдържа абстрактни методи, които връщат DAO обекти за всяка същност, която искате да използвате във вашето приложение.

Използване на базата данни в приложението:

Сега може да се получи инстанция на базата данни в приложението, като се използват създадените методи на RoomDB. Използвайки тези методи, могат да се извършват различни операции с базата данни, като вмъкване, извличане и актуализиране на данни.

Общо взето, SQLite е мощен инструмент за работа с данни в мобилната разработка, който предоставя бърз и ефективен начин за съхранение, управление и манипулиране на данни във вашите Android приложения. Използването на SQLite във вашия код може да подобри производителността и стабилността на вашето приложение, като в същото време осигурява надеждност и сигурност на данните.

2.1.4 Използвани сензори и услуги

2.1.4.1 TYPE_STEP_COUNTER

Сензорът TYPE_STEP_COUNTER е специализиран сензор, наличен в Android устройствата, специално създаден за броене на броя стъпки, направени от потребителя. За разлика от другите сензори за движение, сензорът TYPE_STEP_COUNTER предоставя точно броене на стъпките, като брои стъпките непрекъснато, дори когато екранът на устройството е изключен или приложението не работи активно.

За да се интегрира използването на сензора TYPE_STEP_COUNTER в Android приложение, се следват следните стъпки:

Проверка за наличност на сензора:

Преди да използват сензора TYPE_STEP_COUNTER, разработчиците трябва да проверят дали устройството поддържа този вид сензор. Това може да се направи програмно, като се провери списъка със сензори на устройството чрез класа SensorManager.

Регистриране на слушател на сензора:



След като наличието на сензора е потвърдено, приложението регистрира слушател на сензора, за да получава актуализации от сензора TYPE_STEP_COUNTER. Това се случва постоянно, тъй като отброяването на стъпки е в Service клас, който винаги стои активен.

Обработване на събитията на сензора:

Когато стъпките се откриват от сензора, системата предоставя събития от сензора на регистрирания слушател. Приложението след това обработва тези събития, за да актуализира информацията за броя стъпки и да я покаже на потребителя.

Отписване на слушателя на сензора: За да се пести живот на батерията и ресурсите на системата, важно е да се отпише слушателят на сензора, когато той вече не е нужен. Тъй като консумацията на батерията от приложението е достатъчно ниска, използвайки само един сензор и не използвайки интернет ресурси освен веднъж на ден, това се случва единствено когато потребителя излезе от акаунта си.

2.1.4.2 GMS LocationServices

Location Services на Google Play Services (GMS) предоставят различни функционалности, свързани с определянето на местоположението на устройството. Те използват различни източници на данни, като GPS, Wi-Fi, мобилни мрежи и сензори на устройството, за да определят текущото местоположение на потребителя. Ето някои от основните функции на Location Services на GMS:

Получаване на текущо местоположение:

Location Services предоставят API, които позволяват на приложенията да получат текущото местоположение на устройството в реално време. Това може да бъде полезно за различни приложения като приложения за навигация, социални мрежи, местоположени базирани услуги и други.

Определение на географска област:

Помощта на Location Services приложенията могат да определят кога устройството влиза или излиза от определена географска област (геофенс). Това е полезно за сценарии като тригериране на събития при влизане или излизане от определена зона.

Проследяване на местоположението на потребителя:

Location Services предоставят възможности за проследяване на местоположението на потребителя в продължение на време. Това е полезно за приложения като приложения за фитнес, доставка на стоки, таксиметрови услуги и други.



Разширена точност и батерийна ефективност: Location Services оптимизират процеса на определяне на местоположението, като използват различни източници на данни и алгоритми за подобряване на точността и ефективността. Те също така предоставят възможности за управление на батерията, като например различни режими на проследяване и ограничаване на използването на GPS при ниска консумация на енергия.

Интеграция с други услуги на Google: Location Services се интегрират с други услуги на Google, като например Google Maps, което позволява на разработчиците да създават мощни и интелигентни приложения, използващи местоположението като основна функционалност.

Тези функционалности на Location Services на GMS предоставят инструментите и възможностите, необходими за лесното създаване на услугата за дейности, в която се използва местоположението на устройството на потребителя като спрямо дистанцията преминава и времето което е отнело, услугата изчислява моментната скорост, а приложението я сумира и разделя за да открие средната скорост и да изчисли изгорените калории.

2.1.4.3 NOTIFICATION_SERVICE

NOTIFICATION_SERVICE е системна услуга в Android операционната система, която позволява на приложенията да създават и управляват известия (notifications) на устройството. Тези известия могат да бъдат показани на потребителя, за да го уведомят за различни събития, като нови съобщения, събития от календара, завършване на изтегляне на файл и други важни събития.

Създаване на известия: Приложенията могат да използват NOTIFICATION_SERVICE, за да създават различни видове известия, като текстови съобщения, икони, звукови сигнали и други елементи, които се показват на екрана на устройството. Приложението използва един и същи вид нотификация в две различни услуги.

Персонализация на известията: Разработчиците могат да персонализират известията, като добавят различни опции за действия (action buttons), групиране на известията, промяна на цветове, звукови ефекти и др., за да направят известията по-атрактивни и функционални за потребителя. Известията на приложението се обновяват при различни получени данни от TYPE_STEP_COUNTER и от GMS Location Services.



Управление на приоритета на известията: NOTIFICATION_SERVICE позволява на разработчиците да управляват приоритета на известията, което позволява на потребителя да настрои важността на различните известия според техните предпочитания и нужди.

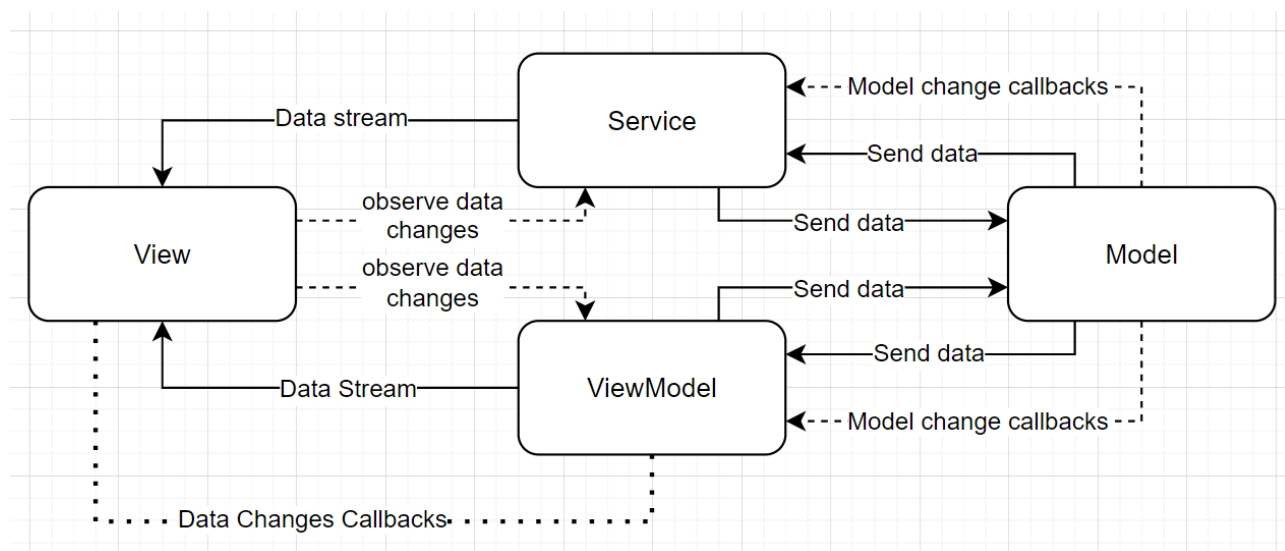
Интерактивни известия: Приложенията могат да създават известия с интерактивни функции, като например възможност за отговаряне на съобщения, пускане на музика, контроли за възпроизвеждане на медийно съдържание и други, които позволяват на потребителя да взаимодейства директно с известията.

Уведомяване за важни събития: NOTIFICATION_SERVICE позволява на приложенията да уведомяват потребителите за важни събития, като например нови съобщения, обновления на приложения, събития от календара, предупреждения за системни събития и други, които са от значение за потребителя.

Общо взето, NOTIFICATION_SERVICE предоставя създаване и управление на известия в приложението, за да известява потребителите си, че услугите, които са стартирани, работят във фонов режим.

2.2. Софтуерен дизайн

2.2.1 Софтуерна архитектура – MVVM



Архитектурата Model-View-ViewModel (MVVM) е шаблон за проектиране, използван при разработването на софтуер за разделяне на проблемите между потребителския интерфейс (View), бизнес логиката (ViewModel) и данните (Model). Когато се интегрира услуга в MVVM, тя обикновено взаимодейства със слоя Model, за да извлича или манипулира данни от външен източник. Приложението използва MVVM модел с добавен



Service, предоставящ информация, променящ информация и извършващ други операции като създаване и управляване на известия.

2.2.1.1 Model

Моделният слой в софтуерната архитектура представя данните и бизнес логиката на приложението. Той капсулира структурата, поведението и правилата на данните, управлявани от приложението.

Представяне на данни:

В основата си Моделният слой определя как данните са структурирани в приложението. Това включва дефиниране на същности (entity), класове-данни или таблици от базата данни, които представят различните видове данни, използвани от приложението. Същностите обикновено съдържат атрибути, които описват техните свойства. Например в приложение за управление на задачи същността Task (Задача) може да има атрибути като заглавие, описание, срок за изпълнение и др.

Бизнес логика:

Слой "Модел" съдържа и бизнес логиката на приложението, която представлява правилата и операциите, които управляват начина на манипулиране и обработка на данните. Бизнес логиката осигурява целостта, последователността и валидността на данните. Тя налага ограничения, извършва изчисления и прилага бизнес правила, за да гарантира, че данните остават точни и смислени. Например в приложение за електронна търговия бизнес логиката може да включва правила за изчисляване на разходите за доставка, прилагане на отстъпки или валидиране на информацията за плащане.

Достъп до данни:

Слоят на модела взаимодейства с източници на данни, като например бази данни, уеб услуги или API, за да извлича, съхранява и манипулира данни. Той абстрахира подробностите за достъпа до данни, като осигурява последователен интерфейс за другите слоеве на приложението.

Като цяло моделният слой играе важна роля в архитектурата на софтуера, като определя структурата на данните, бизнес логиката и механизмите за достъп до данни на приложението. Той действа като гръбнак на приложението, като гарантира, че данните се управляват ефективно и последователно в цялата система.



2.2.1.2 View (UI Layer)

Слоят View в софтуерната архитектура представлява потребителският интерфейс (UI) на приложението. Той отговаря за представянето на данните на потребителя и за улавянето на потребителския вход.

Компоненти на потребителския интерфейс:

Слоят View се състои от различни компоненти на потребителския интерфейс, като бутони, текстови полета, изображения, списъци и други визуални елементи, с които потребителите взаимодействат. Тези компоненти обикновено се реализират с помощта на специфични за платформата технологии, като например XML файлове за оформление при разработване на Android или HTML, CSS и JavaScript при разработване на уеб.

Разположения на екрана:

Изгледите са организирани в екранни оформления, които определят подредбата и външния вид на компонентите на потребителския интерфейс на екрана. Екранните оформления определят позиционирането, оразмеряването и стилизирането на елементите на потребителския интерфейс, за да се създаде визуално привлекателен и удобен за потребителя интерфейс. Разположенията могат да бъдат статични или динамични, в зависимост от това дали се адаптират към различни размери, ориентации или типове устройства на екрана.

Логика на потребителския интерфейс:

Слоят View съдържа логиката на потребителския интерфейс, която управлява поведението и взаимодействието на компонентите на потребителския интерфейс. Логиката на потребителския интерфейс обработва събитията за въвеждане на данни от потребителя, като например натискане на бутон, въвеждане на текст или жестове, и реагира съответно на тях. Тя може да включва актуализиране на дисплея въз основа на действията на потребителя, показване или скриване на елементи, навигация между екрани или показване на обратна връзка на потребителя.

Обвързване на данни:

Свързването на данни е техника, използвана за свързване на слоя View с основните данни, представени от слоя Model. Тя позволява компонентите на потребителския интерфейс да се свързват директно с обекти от данни, като автоматично актуализират потребителския интерфейс при промяна на данните. Свързването на данни намалява необходимостта от ръчно актуализиране на елементите на потребителския интерфейс и помага за синхронизиране на потребителския интерфейс с основните данни.



Наблюдаеми обекти и събития:

Изгледите често наблюдават промени в данните или реагират на събития, предизвикани от взаимодействието с потребителя или системни събития. Наблюдаемите елементи, като например LiveData при разработката на Android или наблюдаемите елементи в реактивните програмни рамки, позволяват на изгледите да се актуализират реактивно в отговор на промените в основните данни. Събитията, като натискане на бутон или жестове с докосване, се обработват от слушатели на събития или функции за обратно извикване, прикрепени към компонентите на потребителския интерфейс.

Достъпност и използваемост:

Слоят View отговаря за осигуряването на достъпност и използваемост на потребителския интерфейс от всички потребители, включително тези с увреждания или специални нужди. Функциите за достъпност, като например екранни четци, гласови команди или алтернативни методи за въвеждане, могат да бъдат приложени, за да се подобри използваемостта на приложението за всички потребители.

Като цяло слойът View играе решаваща роля в софтуерната архитектура, като осигурява интерфейса, чрез който потребителите взаимодействат с приложението. Той представя информацията на потребителя по смислен начин и позволява на потребителите да взаимодействат с функционалността на приложението ефективно и интуитивно.

2.2.1.3 View Model

Слоят ViewModel, известен още като Presentation layer, служи като посредник между View (потребителски интерфейс) и Model (данни) в архитектурата MVVM (Model-View-ViewModel). Той съдържа логиката на представянето и излага данните на View за показване. Ето едно подробно обяснение на слоя ViewModel:

Логика на представянето:

ViewModel съдържа логиката на представяне на приложението. Той капсулира поведението и операциите, необходими за подготовка на данните за показване в потребителския интерфейс. Логиката на представяне включва форматиране на данни за показване, като например преобразуване на времеви маркери в четивни за човека дати, форматиране на валута или локализиране на текст за различни езици. Тя може също така да включва обработка на взаимодействията с потребителя, като например натискане на бутон или жестове, и координиране на подходящи действия в отговор на тези взаимодействия.

Излагане на данни:



Моделите ViewModels излагат данни на слоя View за показване. Те предоставят свойства или методи, които представят данните, необходими на компонентите на потребителския интерфейс за визуализиране на потребителския интерфейс. Експонираните данни могат да идват от слоя Model, например същности, извлечени от база данни, или данни, получени от уеб услуга. ViewModels абстрахират подробностите за извличане и манипулиране на данни от слоя View, като осигуряват чисто разделяне на проблемите и насърчават модулността и възможността за тестване.

Осъзнаване на жизнения цикъл (Lifecycle awareness):

ViewModel-ите са изгледи са компоненти, които се съобразяват с жизнения цикъл при разработката на Android. Те оцеляват при промени в конфигурацията, като например завъртане на екрана, и се запазват, докато свързаният контролер на потребителския интерфейс (дейност или фрагмент) е жив. Това гарантира, че данните остават налични и последователни при промени в конфигурацията, което подобрява потребителското изживяване и предотвратява загубата на данни.

Модел на наблюдател:

Моделите на изгледа често използват модела на наблюдателя, за да уведомяват слоя на изгледа за промени в основните данни. Те излагат наблюдаеми свойства, като например обектите LiveData при разработката на Android, които позволяват на Views да наблюдават промените в данните и съответно да актуализират потребителския интерфейс. По този начин View се отделя от ViewModel, което позволява реактивни актуализации на потребителския интерфейс, без да се свързват директно компонентите на потребителския интерфейс с източника на данни.

Управление на състоянието:

Моделите ViewModel управляват състоянието на компонентите на потребителския интерфейс в слоя View. Те съхраняват информация за състоянието на потребителския интерфейс, като например дали трябва да се покаже индикатор за зареждане или дали трябва да се покаже съобщение за грешка. Управлението на състоянието помага да се поддържа последователно състояние на потребителския интерфейс при различни взаимодействия с потребителя и гарантира, че потребителският интерфейс отразява текущото състояние на основните данни.

Тестване:

ViewModels са лесно тестваеми компоненти, тъй като съдържат чиста логика на представянето и нямат зависимост от класовете на рамката на Android. Могат да бъдат



написани тестове на модули, за да се провери поведението на ViewModels независимо от слоя View, като се гарантира, че данните се обработват и излагат правилно за показване.

Като цяло слойът ViewModel играе важна роля в софтуерната архитектура, като служи като мост между View и Model. Той дава възможност за разделяне на проблемите, насърчава повторната използваемост и възможността за тестване и улеснява създаването на поддържани и мащабируеми приложения.

2.2.1.4 Service layer

Слойът на услугите в софтуерната архитектура представлява външен източник на данни или интерфейс към външни системи, като например уеб услуги, API или отдалечени бази данни. Ето едно подробно обяснение на слоя Service:

Външен източник на данни:

Слойът Service действа като портал към външни източници на данни, осигурявайки достъп до данни, съхранявани извън локалната среда на приложението. Външните източници на данни могат да включват уеб сървъри, базирани в облака бази данни, API на трети страни или други отдалечени системи, които излагат данни или функционалност по мрежата.

Достъп до данни и комуникация:

Услугите улесняват достъпа до данни и комуникацията между приложението и външните системи. Те предоставят методи или API за извличане, изпращане, актуализиране или изтриване на данни от външния източник. Достъпът до данни може да включва извършване на HTTP заявки, изпращане и получаване на данни в различни формати (например JSON или XML) или използване на специфични протоколи за комуникация (например REST или SOAP).

Абстракция и капсулиране:

Слойът на услугите абстрахира детайлите на взаимодействието с външни системи, като капсулира сложността на мрежовата комуникация и сериализирането на данните. Чрез капсулирането на тази функционалност услугите осигуряват чист интерфейс за взаимодействие с другите слоеве на приложението, като скриват подробностите за изпълнение и насърчават модулността и поддръжката.

Удостоверяване и оторизация:

Услугите често управляват механизмите за удостоверяване и оторизация, необходими за сигурен достъп до външния източник на данни. Това може да включва прилагане на протоколи за удостоверяване, като например OAuth или API ключове, и проверка на



потребителските разрешения или права на достъп, преди да се разреши достъпът до данни или тяхната модификация.

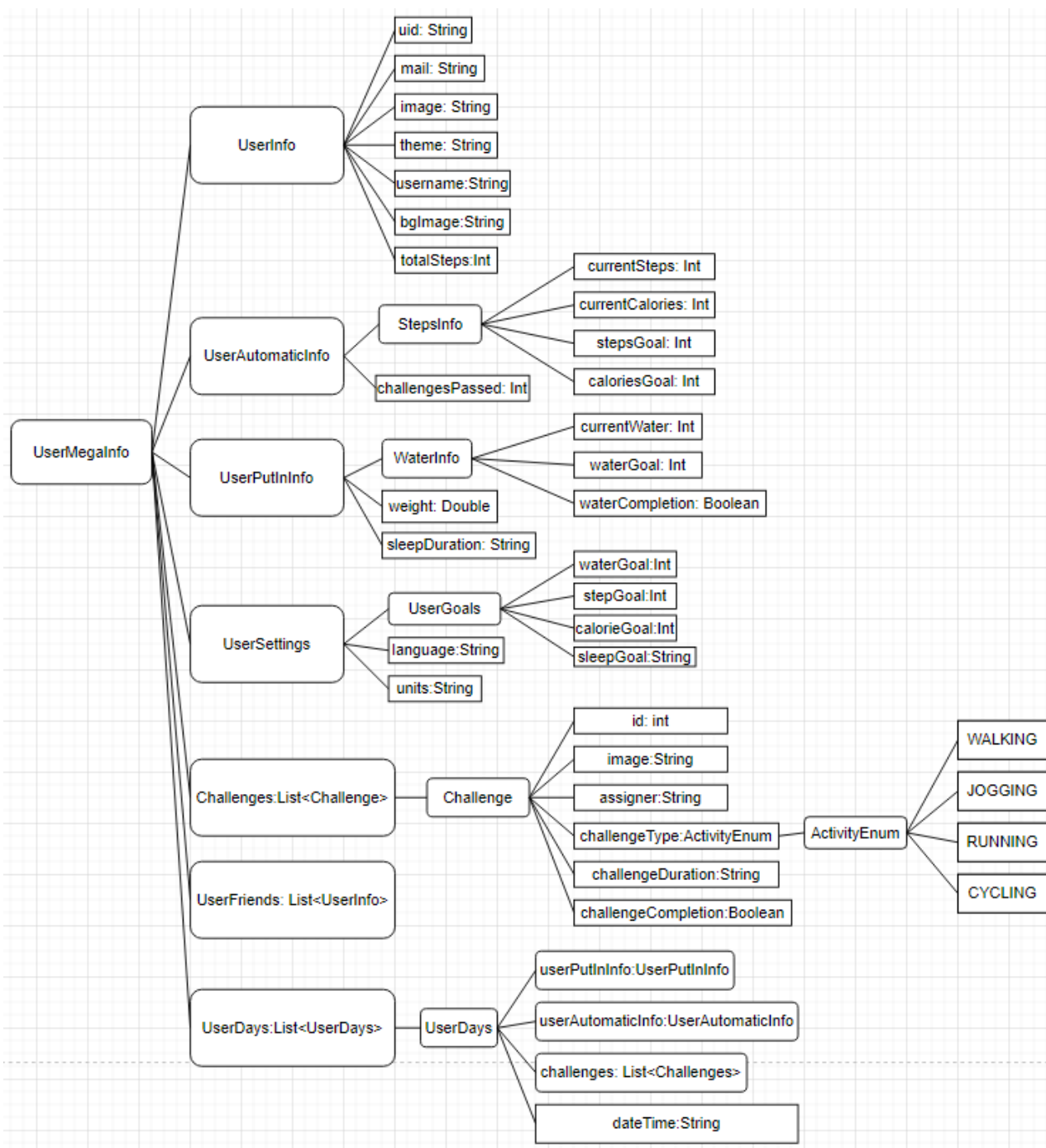
Кеширане и оптимизиране на производителността:

Услугите могат да включват механизми за кеширане с цел подобряване на производителността и намаляване на латентността чрез локално съхраняване на често достъпвани данни. Стратегиите за кеширане могат да помогнат да се сведе до минимум броят на заявките, направени към външния източник на данни, и да се подобри отзивчивостта чрез обслужване на кешираните данни, когато са налични.

Като цяло слойът на услугите играе важна роля в софтуерната архитектура, като осигурява достъп до външни източници на данни и улеснява комуникацията между приложението и отдалечените системи. Той абстрахира сложността на мрежовата комуникация, удостоверяването и обработката на грешки, като позволява на приложението да взаимодейства с външни източници на данни по сигурен, ефективен и надежден начин.



2.2.2 Дизайн на базата данни



Приложението притежава две бази данни – една локална и една облачна, с цел да се намали използвания интернет от приложението, докато се запазва възможността да се сменят устройствата си без да се губят данните в акаунта.

И двете бази данни са обектно-базирани. Структурата на базата данни е дървовидна, започваща от UserMegaInfo и разклоняваща се до края си.



Обектът **UserMegaInfo** съдържа в себе си обектите:

UserInfo – основната информация на потребителя, тази която се събира най-често и най-бързо, заради простотата на обекта

UserAutomaticInfo – информацията на потребителя, която устройството автоматично записва. Това са дневните крачки и калории. Съдържа също и дневните цели на потребителя и броя изпитания които е изминал за деня.

UserPutInInfo – информацията, която потребителя ръчно въвежда всеки ден, като колко вода е изпил, колко тежи или потвърждението за засечен период на сън от устройството.

UserSettingsInfo – съдържа настройките на акаунта на потребителя – измерителните им предпочитания (kg/lbs, km/mile) и цели за активност.

Challenges – съдържа лист от обекта Challenge с информацията на изпитанията, които потребителя е получил за деня. Кой ги е задал, какъв вид са, тяхното id, продължителност и дали изпитанието е изминато или не.

UserFriends – съдържа лист от обекта UserInfo. Този лист е листа с приятели на потребителя и тъй като локалната база данни не го пипа и не се нуждае от него, облачната база данни не го синхронизира спрямо локалната.

UserDays – съдържа лист от обекта UserDays, събиращ информацията от UserAutomaticInfo, UserPutInInfo и Challenges дневно, както и датата на създаването на обекта. Информацията от този лист се използва за визуализацията на прогреса на потребителя.

Всеки ден, в 23:59:30, приложението събира дневната информация в обекта UserDays, вкарва я в листа UserDays, нулира я от локалната база данни за да може да се започне на следващия ден от нулата и се синхронизира с облачната база данни.

При излизане от акаунта, локалната база данни праща данните си към облачната, след което се изчиства. При влизане в друг акаунт, приложението взема информацията от облачната база данни и я дава на локалната.

При опит на потребителя за използва функция, за която е нужен достъп към облачната база данни, без наличност на интернет, заявката му се изхвърля.

При изтриване на акаунт, информацията за акаунта се трие както от локалната база данни RoomDB, така и от облачната Firebase Realtime Database и от услугата за удостоверяване Firebase Authentication.



2.3. Удостоверяване на потребителя

Firebase Auth е част от платформата Firebase, която предоставя облачни услуги за разработка на приложения. Firebase Auth е специализирана услуга, която предоставя инструменти и инфраструктура за управление на идентичността и удостоверяването на потребителите в приложенията.

Управление на идентичността:

Firebase Auth управлява идентичността на потребителите в приложението, като предоставя възможност за регистрация и управление на потребителски профили.

Регистрация и вход:

Потребителите могат да създадат акаунти в приложението чрез различни методи на регистрация, като например с електронна поща и парола, номер на мобилен телефон, социални мрежи като Google, Facebook, Twitter и други. След като потребителите са създали акаунти, те могат да влизат в приложението с помощта на своите идентификационни данни.

Управление на сесиите:

Firebase Auth управлява сесиите на потребителите и предоставя възможности за управление на влизането и излизането от сесиите.

Интеграция с други услуги на Firebase:

Интегрирана е Firebase Auth една от другите услуги на Firebase – Firebase Realtime Database, за да се предостави авторизиран достъп до данни за потребителите.

2.4. CRUD заявки

CRUD означава създаване (create), четене (read), актуализиране (update) и изтриване (delete). Това е набор от основни операции, използвани в базите данни и системите за съхранение на данни. За дефиниране и използване на CRUD заявки се създава отделен интерфейс със заявките за Room (фиг. 2.4.1) и за Firebase (фиг. 2.4.2). При Firebase се създава и клас за имплементация на интерфейса (фиг. 2.4.3).

```
@Dao
interface UserDao {
    @Maxiankata
    @Insert(onConflict = OnConflictStrategy.REPLACE)
    fun saveUser(userMegaInfoEntity: UserData)
    @Maxiankata
    @Query("SELECT * FROM UserData")
    fun getEntireUser(): UserData
    new *
    @Query("UPDATE UserData SET UserInfo = :updatedUserInfo")
    suspend fun updateUserInfo(updatedUserInfo: UserInfo)
    @Maxiankata
    @Query("DELETE FROM UserData")
    fun dropUser()
}
```

Фигура 2.4.1

```
Maxiankata +1 *
interface AuthInterface {
    @Maxiankata
    suspend fun setUser(email: String, username: String, uid: String)
    new *
    suspend fun getUserInfo(uid: String): UserInfo?
    new *
    suspend fun updateUserInfo(userInfo: UserInfo)
    new *
    suspend fun deleteCurrentUser()
}
```

Фигура 2.4.2



```
35 class AuthImpl : AuthInterface {  
    Maxiankata *  
36 override suspend fun setUser(email: String, username: String, uid: String) {...}  
    new *  
45 override suspend fun getUserInfo(uid: String): UserInfo? {...}  
    Maxiankata *  
54 override suspend fun updateUserInfo(userInfo: UserInfo) {...}  
    new *  
60 override suspend fun deleteCurrentUser() {...}
```

Фигура 2.4.3

Приложението използва CRUD заявки по следният начин:

2.4.1 Create

2.4.1.1 Create в RoomDB

За да създаване на база данни в RoomDB, се дефинират класове entity (същности), представляващи таблици от базата данни, и се анотират с `@Entity`. След това се създава клас за база данни, разширяващ `RoomDatabase`, анотира се с `@Database`, посочват се същностите и версията и се усигорява абстрактен метод, връщащ интерфейса DAO. Накрая се инициализира инстанция на базата данни, като се използва `Room.databaseBuilder()`, и се

```
Maxiankata  
@Database(entities = [UserData::class], version = 3)  
@TypeConverters(GsonTypeConverter::class)  
abstract class UserDB : RoomDatabase(){  
    Maxiankata  
    abstract fun dao(): UserDao  
}
```

осъществява достъп до DAO, за да се извършват операции с базата данни.

Този фрагмент от код дефинира клас за база данни Room, наречен UserDB.

Ето обяснение на всяка част:

`@Database(entities = [UserData::class], version = 3)`: Тази анотация се използва за деклариране на клас база данни Room. Тя приема два параметъра:

`entities`: Посочва същностите, свързани с тази база данни. В този случай тя включва само един обект, `UserData::class`.



version: Посочва номера на версията на базата данни. Ако промените схемата на базата данни, трябва да увеличите този номер, за да задействате миграция.

@TypeConverters(GsonTypeConverter::class): Тази анотация се използва за определяне на TypeConverters за базата данни. TypeConverters се използват за преобразуване на сложни типове данни (като потребителски класове) в типове, които Room може да запази в базата данни. В този случай GsonTypeConverter::class е персонализиран TypeConverter, който преобразува сложни типове данни в JSON низове с помощта на библиотеката Gson.

abstract class UserDB : RoomDatabase(): Този ред дефинира класа UserDB като абстрактен клас, който разширява RoomDatabase. Този клас служи като основна входна точка за базата данни Room.

abstract fun dao(): UserDao: Тази абстрактна функция декларира метод, наречен dao(), който връща инстанция на UserDao. UserDao е интерфейс, който предоставя методи за извършване на CRUD операции.

```
private lateinit var db: UserDB
@Maxiankata
fun getDatabaseInstance(): UserDB {
    val context = MyApplication.getContext()
    if (!::db.isInitialized) {
        db = Room.databaseBuilder(
            context.applicationContext, UserDB::class.java, name: "user-base"
        ).build()
    }
    return db
}
```

Този фрагмент дефинира единичен клас UserDB за достъп до базата данни Room. Той инициализира екземпляра на базата данни by lazy* с помощта на Room.databaseBuilder(), като посочва контекста на приложението, класа на базата данни UserDB и името на базата данни "user-base", осигурявайки един екземпляр на базата данни през целия жизнен цикъл на приложението.

* "Lazy initializing" означава, че обектът се създава само при първия достъп до него или при нужда от него, а не в момента на деклариране.

2.4.1.2 Create във Firebase Realtime Database

Създаването на база данни в Firebase Realtime Database включва достъп до определено място в структурата на базата данни и задаване на стойност там. В предоставения фрагмент



функцията setUser постига това, като получава препратка към възела "user" в базата данни, използвайки UID на текущия потребител като част от пътя, и след това задава стойност на това място.

```
Maxiankata +1 *
class AuthImpl : AuthInterface {
    Maxiankata *
    override suspend fun setUser(email: String, username: String, uid: String) {
        Firebase.database.getReference(path: "user/${Firebase.auth.currentUser?.uid}").setValue(
            UserMegaInfo(
                UserInfo(
                    username, uid, image: "", email
                )
            )
        )
    }
}
```

Функцията setUser, маркирана като suspend, замества подобна функция, дефинирана в интерфейс, свързан с асинхронен достъп до функции на Firebase. Тя ефективно актуализира потребителските данни в базата данни в реално време на Firebase, като използва механизмите за суспендиране на асинхронни операции.

Firebase.database.getReference("user/\${Firebase.auth.currentUser?.uid}") извлича конкретно място в базата данни, като динамично конструира пътя въз основа на UID на текущия потребител във възела "user".

setValue(UserMegaInfo(UserInfo(username, uid, "", email)))) методът съхранява информация за потребителя в извлеченото местоположение. Той създава инстанции на UserMegaInfo и UserInfo, като ги инициализира с предоставените данни, преди да ги зададе в базата данни.

2.4.2 Read

Операцията за четене в системите за бази данни извлича данни от определено място или таблица в базата данни, като обикновено се използват механизми за заявки като SELECT. След това извлечените данни могат да бъдат достъпни или обработени от приложението или потребителя.

2.4.2.1 Read в RoomDB

Операцията четене в RoomDB включва заявка към базата данни за извличане на данни от определена таблица или обект. Обикновено това се прави с помощта на методи, предоставени от интерфейса Data Access Object (DAO), като анотации @Query за изпълнение



на SQL заявки или предварително дефинирани методи за достъп до конкретни редове или същности.

```
Maxiankata  
@Query("SELECT userInfo FROM UserData")  
fun getUserInfo(): UserInfo?
```

Тази функция, аотирана с `@Query`, извлича информация за потребителя от таблицата "UserData" в базата данни Room. Тя изпълнява SQL заявката "SELECT userInfo FROM UserData", за да извлече колоната userInfo от таблицата UserData, като връща един екземпляр на UserInfo или null, ако не са намерени данни.

2.4.2.2 Read във Firebase Realtime Database

Операцията четене в базата данни Firebase Realtime Database включва извличане на данни от определено място в базата данни. Обикновено това се прави с помощта на препратка към желаното място и достъп до данните чрез методи като `getValue()`.

```
override suspend fun getUserInfo(uid: String): UserInfo? {  
    return try {  
        val userInfoRef = Firebase.database.getReference(path: "user/$uid/userInfo")  
        val dataSnapshot = userInfoRef.get().await()  
        dataSnapshot.getValue(UserInfo::class.java)  
    } catch (e: Exception) {  
        null  
    }  
}
```

Тук функцията извлича потребителска информация, свързана с определен потребителски идентификатор (uid), от базата данни Firebase Realtime Database. Тя извлича асинхронно данните, съхранявани по пътя "user/{uid}/userInfo", преобразува ги в инстанция на UserInfo с помощта на `getValue()` и ги връща. Ако по време на процеса възникне някакво изключение, функцията връща null.

2.4.3 Update

Операциите за актуализиране при управлението на бази данни включват промяна на съществуващи записи на данни. Тези операции обикновено включват промяна на стойностите на конкретни полета в рамките на даден запис или актуализиране на цели



записи. Актуализациите са от съществено значение за поддържане на точна и актуална информация в базата данни, като често се изпълняват въз основа на определени условия или критерии. Те гарантират, че данните отразяват най-новите промени или корекции, направени от потребителите или приложенията, взаимодействащи с базата данни.

2.4.3.1 Update в RoomDB

В Room операциите за актуализиране променят съществуващите записи в базата данни, като променят стойностите на определени полета. Обикновено това се постига с помощта на методи, дефинирани в интерфейса на Data Access Object (DAO), като например анотациите @Update за актуализиране на същности въз основа на техните първични ключове.

```
Maxiankata
@Query("UPDATE UserData SET userPutInInfo = :updatedUserPutInInfo")
fun updateUserPutInInfo(updatedUserPutInInfo: UserPutInInfo)
```

Този фрагмент дефинира функция, анотирана с @Query, за актуализиране на полето "userPutInInfo" в таблица "UserData". Тя изпълнява SQL UPDATE заявка, която настройва колоната "userPutInInfo" на предоставената стойност updatedUserPutInInfo, което включва промяна на съществуващи записи въз основа на определени критерии или условия.

2.4.3.2 Update във Firebase Realtime Database

Операциите за актуализиране в базата данни Firebase Realtime Database включват промяна на съществуващи данни на определено място в базата данни. Обикновено това се прави чрез задаване на нови стойности за конкретни полета или възли, което задейства актуализации в реално време за всички свързани клиенти, които слушат това местоположение.

```
Maxiankata *
override suspend fun updateUserInfo(userInfo: UserInfo) {
    Firebase.database.reference.child( pathString: "user/${userInfo.uid}/userInfo").setValue(userInfo)
}
```

Тази функция актуализира информацията за потребителя, съхранявана в базата данни Firebase Realtime Database. Тя асинхронно задава стойността на възела "userInfo", разположен по пътя "user/{userInfo.uid}/userInfo", на предоставения обект userInfo, като актуализира данните на потребителя в базата данни.



2.4.4 Delete

Операциите за изтриване в базите данни включват премахване на съществуващи записи на данни, което обикновено се постига чрез команди DELETE в SQL. Тези операции гарантират, че нежеланите или остарелите данни се премахват от базата данни, като се запазва целостта на данните и се оптимизира пространството за съхранение.

2.4.4.1 Delete в RoomDB

В RoomDB операциите за изтриване включват премахване на съществуващи записи от базата данни. Обикновено това се прави с помощта на методи, дефинирани в интерфейса на Data Access Object (DAO), като например анотациите @Delete за изтриване на същности въз основа на техните първични ключове. Изтриването гарантира, че нежеланите или остарелите данни се премахват от базата данни, като се поддържа целостта на данните и се оптимизира производителността.

```
Maxiankata  
@Query("DELETE FROM UserData")  
fun dropUser()
```

Този фрагмент дефинира функция, анотирана с @Query, за изтриване на всички записи от таблицата "UserData" в базата данни Room. Тя изпълнява SQL заявка DELETE, която премахва всички редове от таблицата "UserData", като ефективно изчиства всички съществуващи данни от таблицата.

2.4.4.2 Delete във Firebase Realtime Database

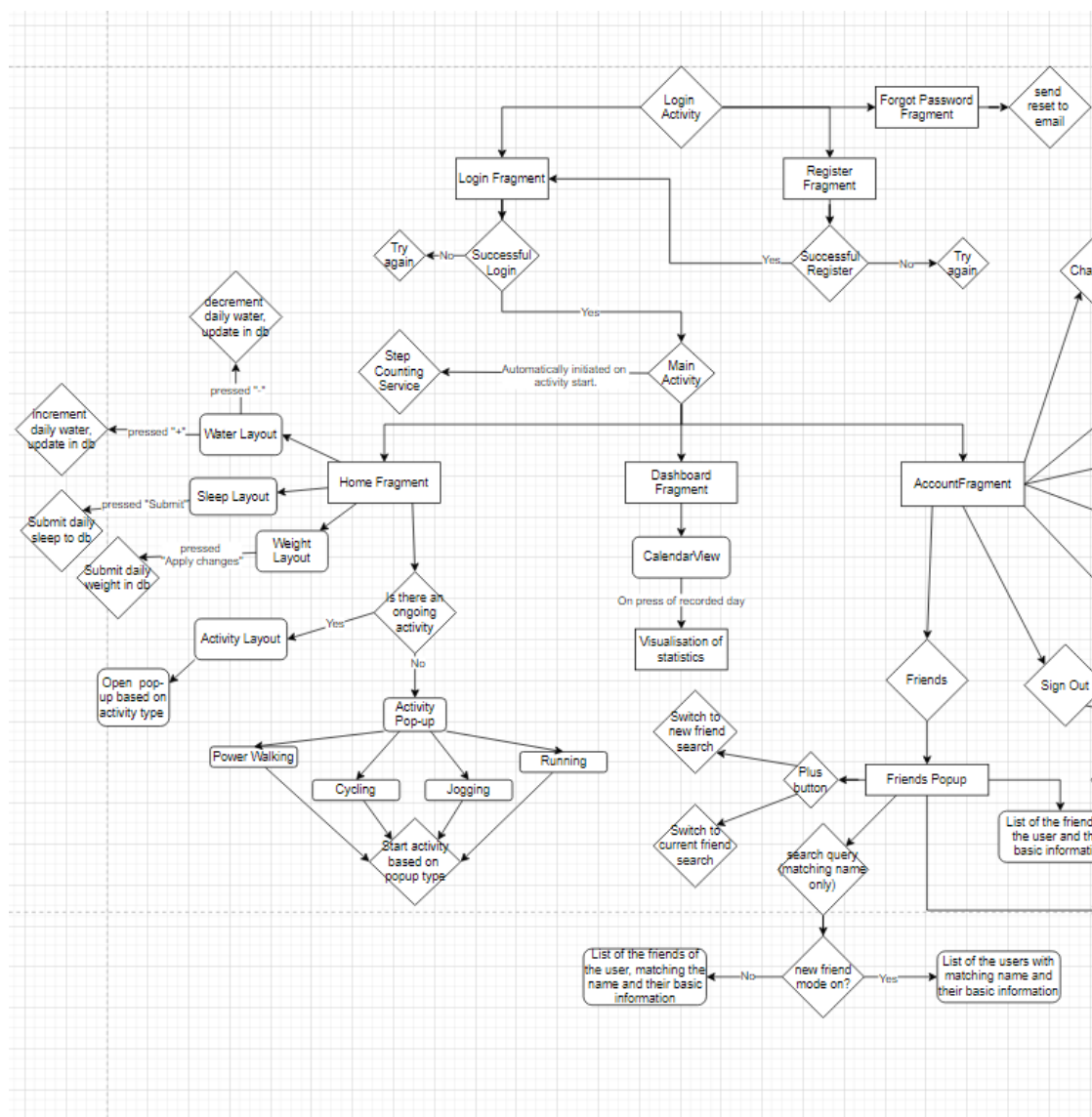
```
new *  
override suspend fun deleteUser() {  
    Firebase.database.getReference( path: "user/${Firebase.auth.currentUser?.uid}").setValue(null)  
    Firebase.auth.currentUser?.delete()?.addOnCompleteListener { task ->  
        if (task.isSuccessful) {  
            signOut()  
        }  
    }  
}
```

Тази функция изтрива данните на текущия потребител и удостоверителните данни за удостоверяване от услугите на Firebase. Тя задава стойността на пътя "user/{currentUserId}" в базата данни в реално време на null, като ефективно премахва данните на потребителя. След



това се извиква delete() на обекта за удостоверяване на текущия потребител, което изтрива потребителския акаунт от Firebase Authentication. Ако изтриването е успешно, се извиква функцията signOut(), за да се отпише потребителят от приложението.

3. Функционалности на приложението



3.1. Регистриране и влизане – Login Activity

Приложението притежава екрани за регистриране и влизане в акаунта на потребителя. В случай че потребителя е забравил паролата си има възможност да я промени.

3.2. Екрани на MainActivity

След влизане в акаунта се стартира MainActivity, последвано от класа, отговарящ за TYPE_STEP_COUNTER сензора, записването на дневни данни и други.



3.2.1 Home

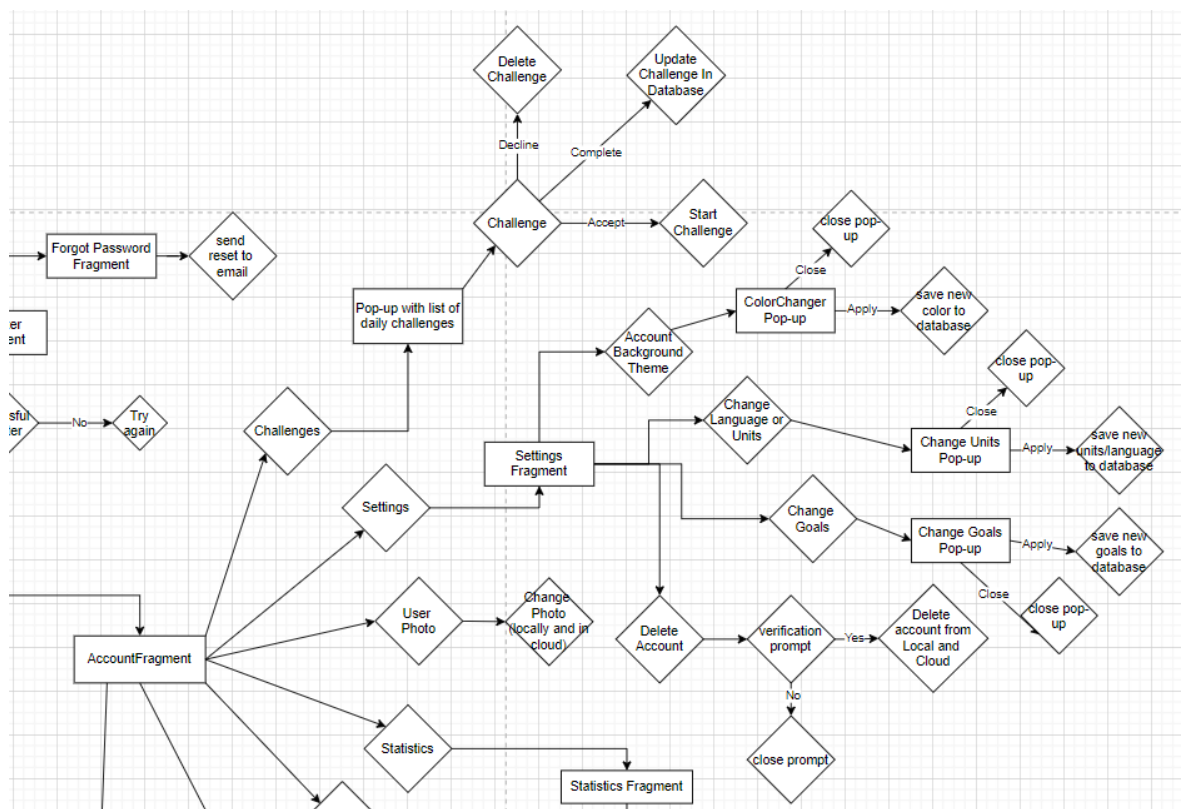
Тук се случва главната дейност на приложението. Визуализират се стъпките и калориите за деня (Приложение, фиг. 6.1). Потребителя записва своите дневни данни за изпита вода, сън и килограми. Потребителя има възможност да стартира активна дейност, избира от 4 типа. От типа зависи как изгорените калории ще бъдат изчислени в края на дейността. При вече стартирана дейност, може да отвори прозорец с информацията на моментната дейност и ако поиска да я спре.

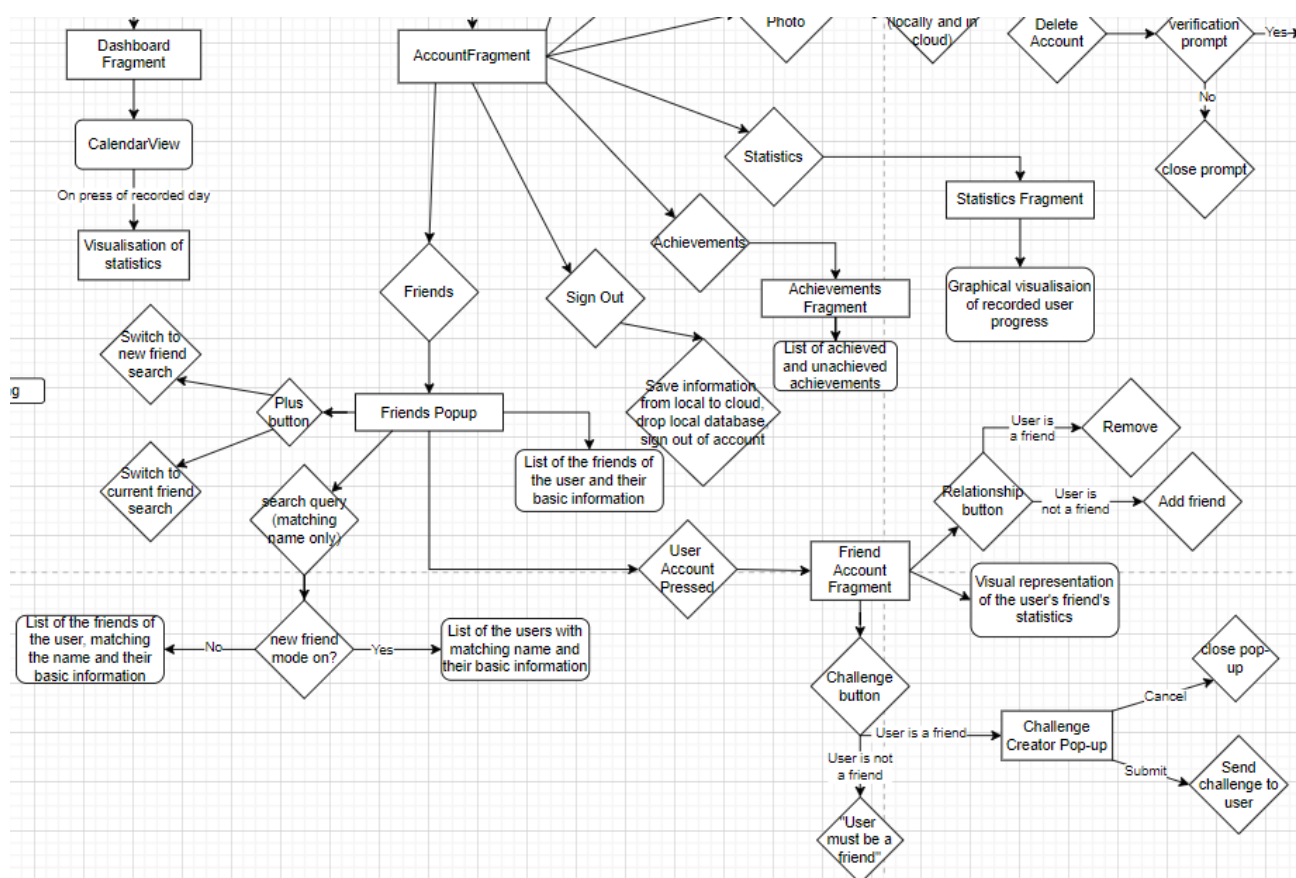
3.2.2 Dashboard

Тук се визуализира информацията от записаните дни. В зависимост от избраната дата на календара се визуализира информация от различен ден

3.2.3 Account

Тук потребителя може да персонализира акаунта си, да приема предизвикателства, да отдава предизвикателства, да праща и приема покани за приятелства, да гледа постиженията или статистиките си, както и тези на приятелите си. Може да променя настройките за акаунта си, да излезе от него или да го изтрие.





3.2.4 Background функционалности

3.2.4.1 StepCounterService

Този клас вид услуга, постоянно активен докато потребителя е в акаунта си, използва TYPE_STEP_COUNTER сензора за да следи стъпките на потребителя. При ходене, на всеки 20-25 стъпки се гори по 1 калория, съответно на всеки 25 стъпки се записва една изгорена калория. За да не се губят данни, но да не се натоварва телефона, на всеки 10 секунди автоматияно се актуализира информацията в базата данни, спрямо взетите стойности от класа.

3.2.4.2 SpeederService

Този клас вид услуга, активен когато има активна дейност, използва GMS Location Services услугата за да следи локацията и скоростта на потребителя. Докато услугата е активна, от стъпките не се броят калории. При завършването на дейността се изчисляват изгорени калории спрямо скоростта на потребителя, теглото му, избраната му дейност и времето ѝ. Изчислява се спрямо мярката MET. MET (метаболичен еквивалент на работа) е



мярка за интензивността на физическата активност. Той измерва количеството енергия, изразходвано от организма по време на дадена дейност. Един MET (MET = 1.0) е приблизително равен на енергията, изразходвана в покой. MET се използва за оценка на физическите усилия и определяне на оптималното тренировъчно натоварване. За да изчислим възможно най-точно калориите, изразходвани от човек използвам формулата:

$$(\text{MET} * 3.5 * \text{тегло} / 200) * \text{време в минути} = \text{KCAL}$$

И тъй като MET мярката се изчислява спрямо литри кислород отдадени, което не мога да взема от потребителя си, използвам таблици направени от учени, специализирани в областта и я задавам като константа.

3.2.4.3 NotificationService

Двете услуги които обясних по-горе използват собствени нотификации.

4. Заключение

4.1. Постигнати цели

Всяка цел поставена на приложението е постигната. Дизайнът е интуитивен, разбираем и грабващ окото. Всеки потребител изживява свой собствен потребителски опит. Изградена е връзка между потребителите за тяхното активно участие и взаимно насърчаване. Приложението работи бързо, не се нуждае от интернет, но пази данните си в облачна база данни за да не бъде лимитиран потребителя към едно устройство. Всички открити нередности по приложението са отстранени.

4.2. Развитие

Приложението може да се развие в много посоки. Като за начало, то е създадено за всекидневния човек и най-важното нещо е да се изгради общност около приложението. Може да се създаде екран за месечна или седмична класация на потребителите, както и такава между приятели. Може при създаването на акаунт, потребителя да избира между нормален акаунт или спортистки, като спортисткия ще присъства в различни класации от нормалните, но ще трябва да държи активността си високо, за да остане акаунта му със статус „Спортист“. Приложението може да партнира със спортни марки и да се добавят реклами, съответно и евтин абонамент за премахването на тези реклами. Може спортистите да имат възможността да създават гилдии, които да се състезават месечно за награди,



получени от партнираните марки. Могат да се създават и месечни или седмични предизвикателства с награди за потребителите на върха.

Потребителите могат да получат възможността да създават или да влизат в акаунтите си чрез Facebook или Google акаунти.

Може в Dashboard екрана на приложението да се добави карта, която следи всекидневния маршрут на потребителя, както и да се имплементира и карта, чрез която могат да се виждат близки потребители, които съответно са позволили да бъдат виждани.

Тъй като приложението е тясно свързано със здравето на човек, имплементирането му към смарт часовник би било напълно умсетно. Потребителите ще имат по-точни сензори от които да се взима и обработва информация, както и възможността да се следи здравето на потребителя по още критерии – например кръвно налягане или пулс.

Въпреки че графичният интерфейс е интуитивен, винаги има посоки на развитие. Потребителите за заключени в тъмна тема, съответно светла тема може да бъде направена. Могат да бъдат добавени по-сложни анимации и да се преправят или премахнат диалогови прозорци.

5. Източници

Kotlin документация <https://developer.android.com/kotlin>

Документация за сензора за стъпки :

<https://developer.android.com/health-and-fitness/guides/basic-fitness-app/read-step-count-data>

https://developer.android.com/develop/sensors-and-location/sensors/sensors_motion

Документация за услуги:

<https://developer.android.com/develop/background-work/services/foreground-services>

<https://developer.android.com/develop/background-work/services/fg-service-types>

Документация за услуги за локация на Google Play:

<https://developer.android.com/develop/sensors-and-location/location/battery>

<https://developer.android.com/develop/sensors-and-location/location/migration>

<https://developers.google.com/android/reference/com/google/android/gms/location/package-summary>

Документация за калкулиране на калории от различни активности. MET:



[https://www.researchgate.net/publication/269927658_mHealth_A_Design_of_an_Exercise Recommendation System for the Android Operating System#pf5](https://www.researchgate.net/publication/269927658_mHealth_A_Design_of_an_Exercise_Recommendation_System_for_the_Android_Operating_System#pf5)

https://www.researchgate.net/figure/Examples-of-MET-values-for-cycling-jogging-and-walking_tbl2_269927658

<https://media.hypersites.com/clients/1235/filemanager/MHC/METs.pdf>

<https://blog.nasm.org/metabolic-equivalents-for-weight-loss>

<https://www.sciencedirect.com/topics/medicine-and-dentistry/metabolic-equivalent>

Използвани библиотеки:

<https://github.com/AACChartModel/AACChartCore-Kotlin?tab=readme-ov-file>

<https://github.com/lospower/CircularProgressBar>

RoomDB документация:

<https://developer.android.com/reference/androidx/room/RoomDatabase>

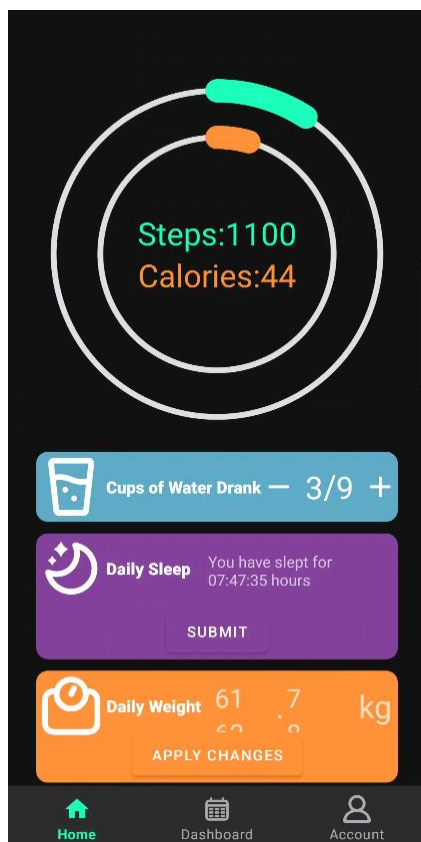
Firebase документация:

<https://firebase.google.com/docs>

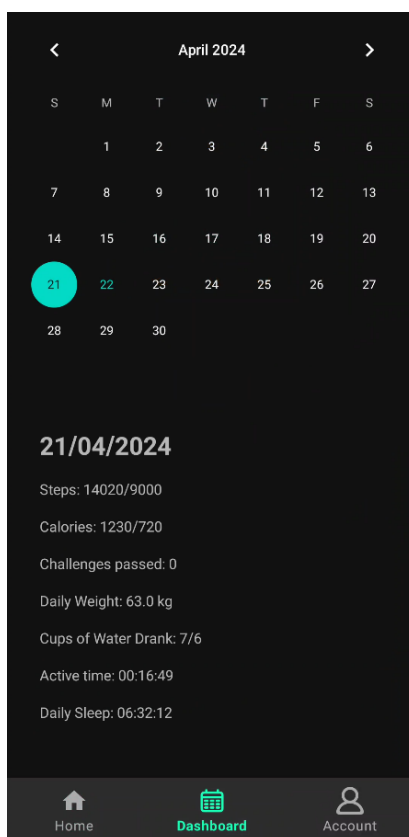
<https://firebase.google.com/docs/database>

<https://firebase.google.com/docs/auth>

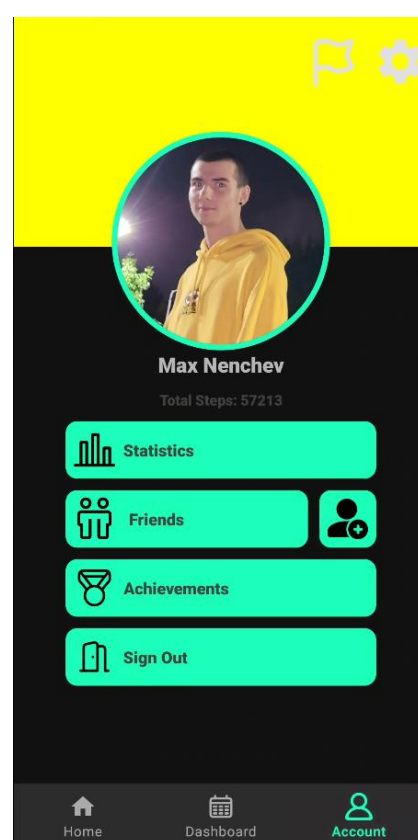
6. Приложение



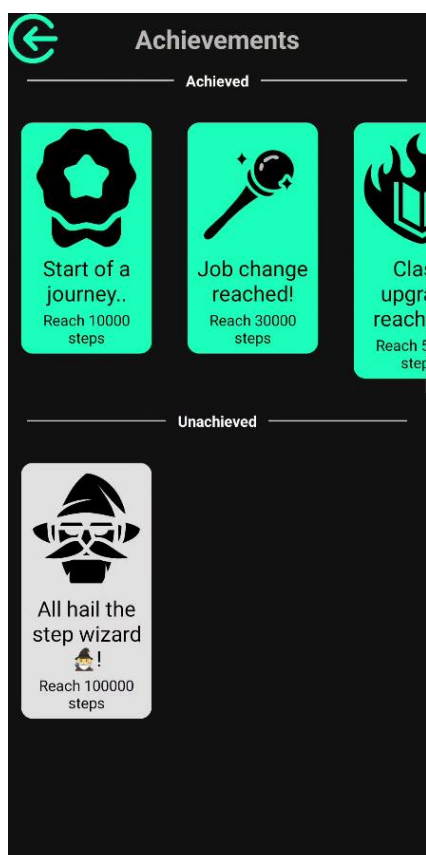
Фигура 6. 1



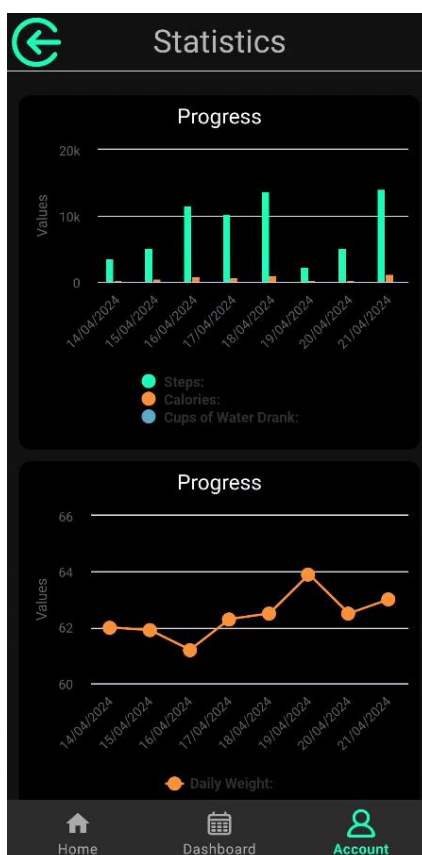
Фигура 6. 2



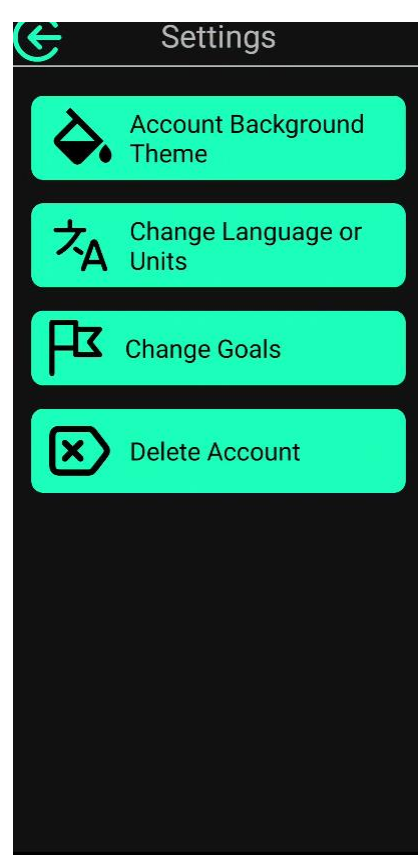
Фигура 6. 3



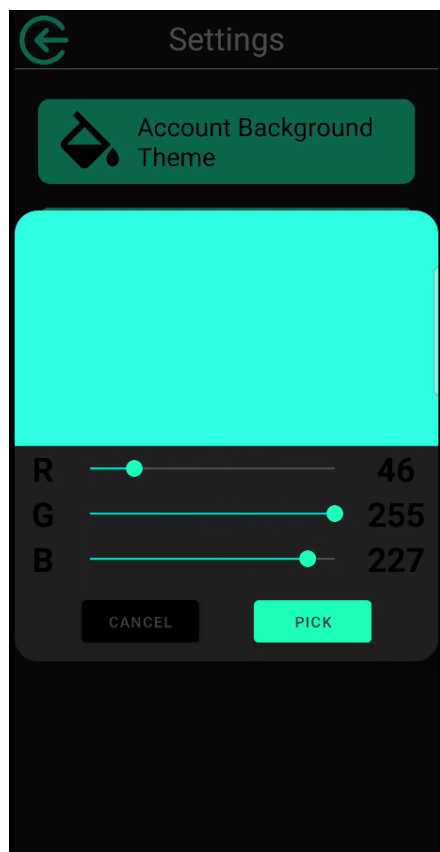
София 2024 Фигура 6. 4



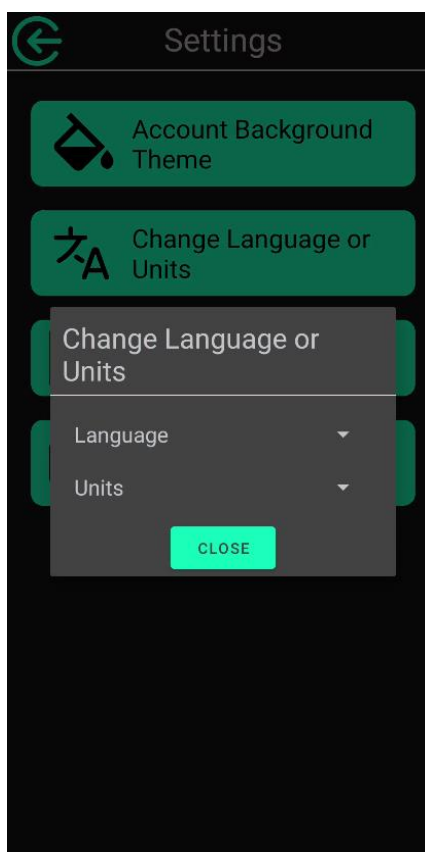
Фигура 6. 5



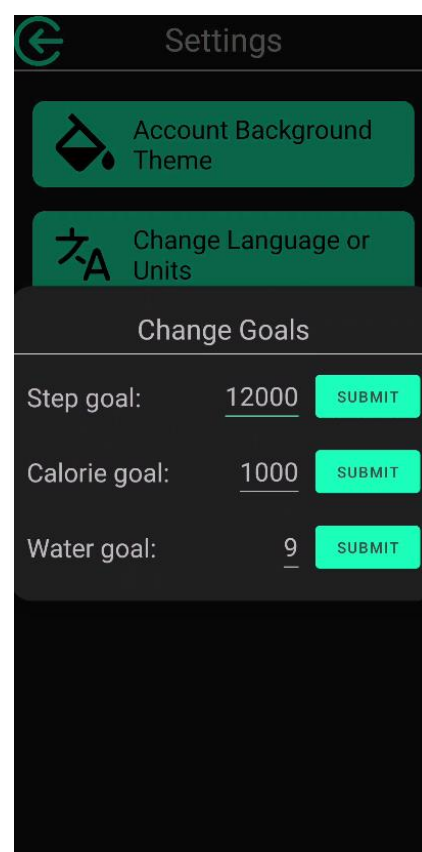
Фигура 6. 6



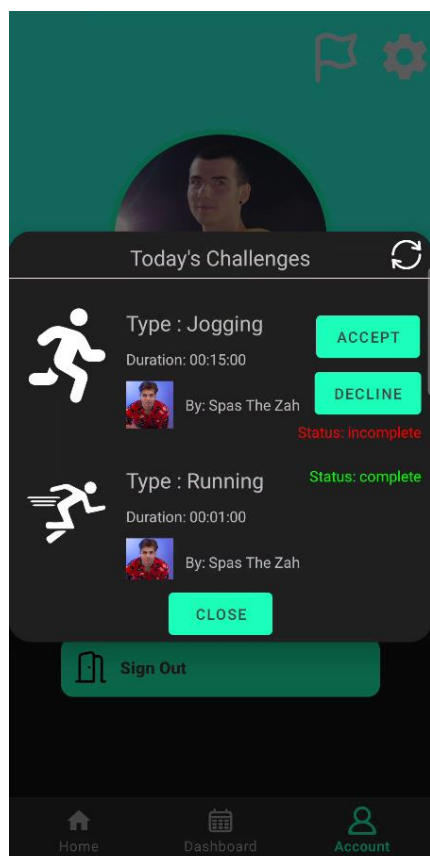
Фигура 6. 7



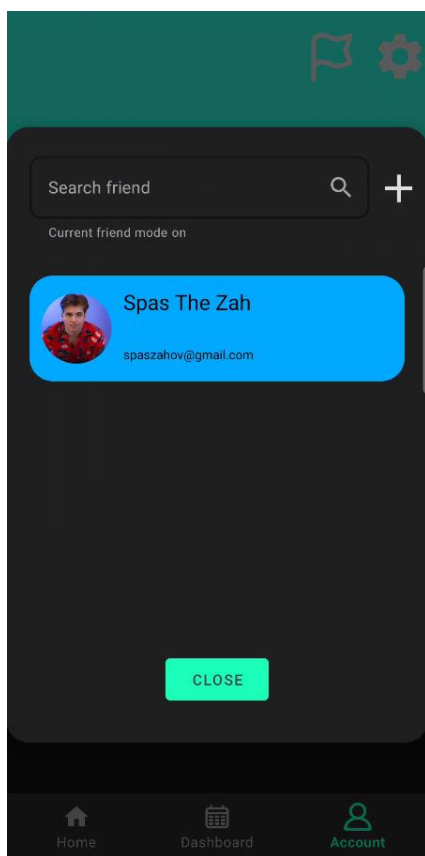
Фигура 6. 8



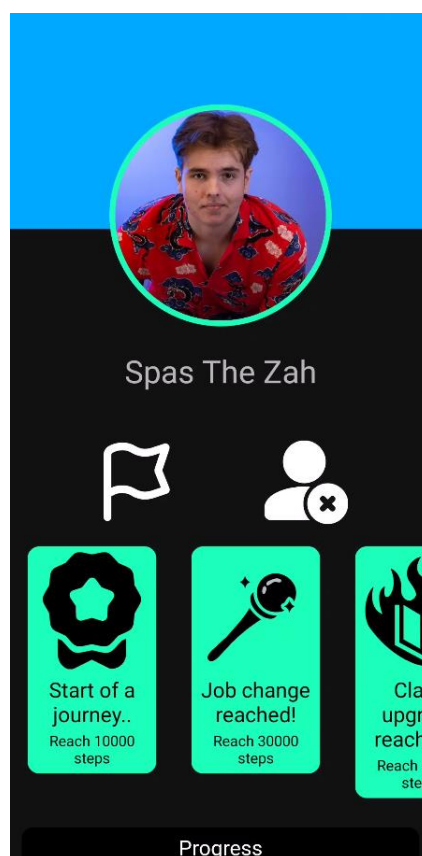
Фигура 6. 9



Фигура 6. 10



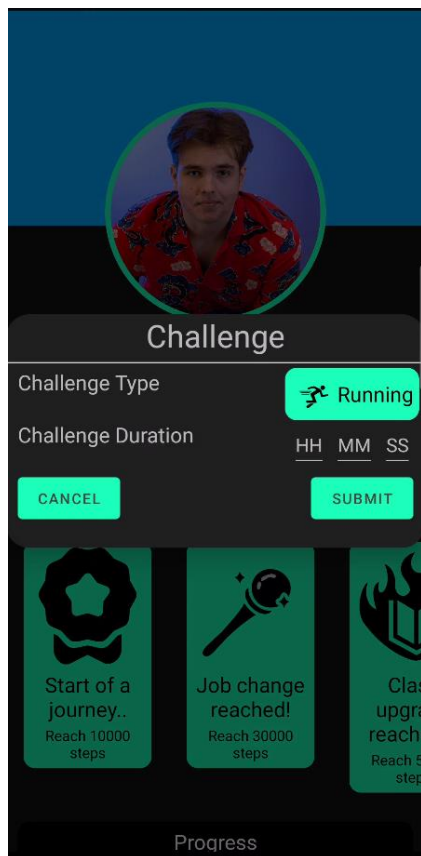
Фигура 6.11



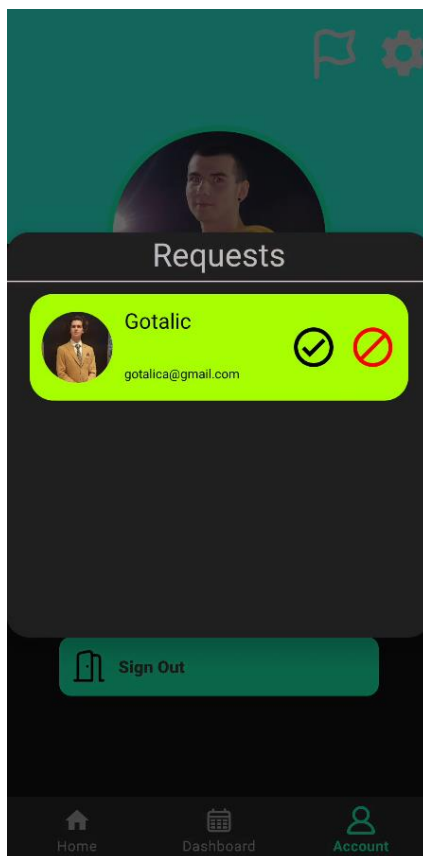
Фигура 6. 12



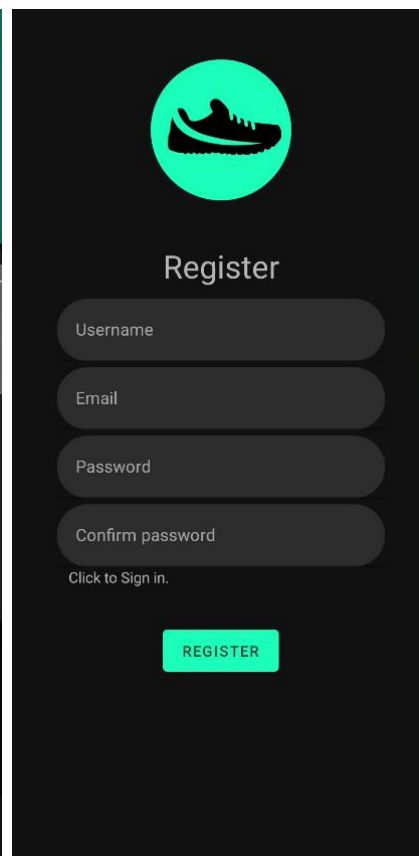
СОФИЙСКА ПРОФЕСИОНАЛНА ГИМНАЗИЯ ПО ЕЛЕКТРОНИКА "ДЖОН АТАНАСОВ"
„Разработване на Android мобилно приложение за следене на здравето на човек“
Максим Спасов Ненчев



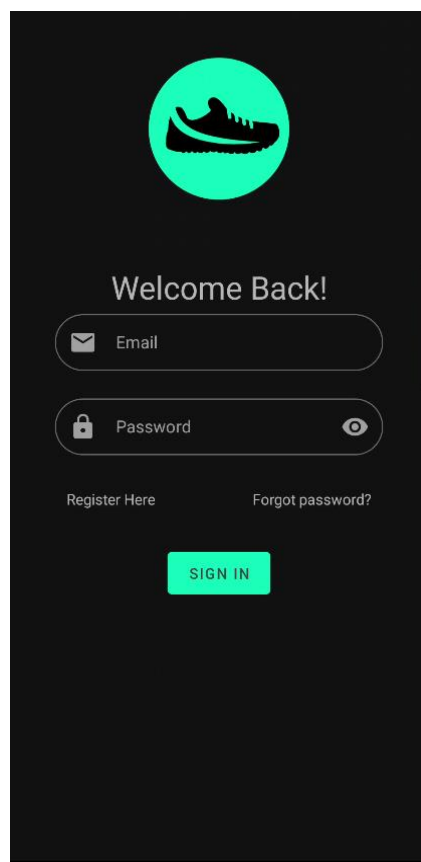
Фигура 6. 13



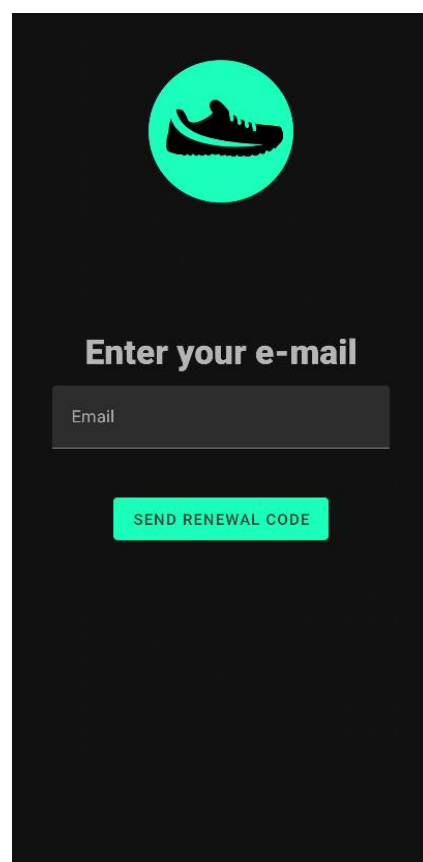
Фигура 6. 14



Фигура 6. 15



Фигура 6. 16



Фигура 6.17